



PRODUCT REQUIREMENT DOCUMENT (PRD)

Project: Sides of Mashriq - Premium Fusion Dining Engine (MERN Stack)

Status: Approved for Development (MERN Transition)

Author: Senior PM & Tech Lead (IQ 200)

1. Executive Summary & Business Objectives

1.1 Product Vision

This project delivers a premium, high-converting digital storefront tailored for an ambitious hybrid restaurant concept: **Sides of Mashriq**. The culinary direction is a high-end fusion of **Italian, Arabic, and Desi (South Asian)** cuisines, balanced with an artisanal **sweets and dessert** showcase. The venue operates as a dual-tier establishment: high-velocity **Casual Fast-Food** alongside a premier **High-End Fine Dining** experience.

1.2 Core Business Goal

The primary objective of this platform is to drive **immediate, high-yield table reservations** directly through the restaurant's owned channel, bypassing expensive third-party reservation systems that erode profit margins.

1.3 Financial Value Proposition

Traditional reservation platforms charge flat-rate or percentage-based fees per booked cover. By routing reservations directly through this zero-commission engine, the restaurant's calculated savings follow this model:

$$\text{Monthly Savings} = N_{\text{covers}} \times (C_{\text{platform}} - C_{\text{direct}})$$

Where:

- N_{covers} is the total number of guest seats booked.
- C_{platform} is the average platform fee per cover (typically \$1.00 to \$2.50).
- C_{direct} is the near-zero marginal processing cost of our serverless or lightweight hosting architecture (utilizing MongoDB Atlas free tier $\approx$ \$0.00).

2. Core User Personas

2.1 The High-Intent Diner (Front-End)

- **Behavior:** Wants to book a premium table instantly or immediately dive into the visual menu to explore the unique cultural fusion of Desi spices, authentic Arabic grills, and Italian pasta options.
- **Need:** Dual immediate-entry points on landing. A booking process that takes fewer than 3 steps and executes in under 1.5 seconds on a standard mobile connection.

2.2 The Front-of-House (FOH) Host (Admin Console)

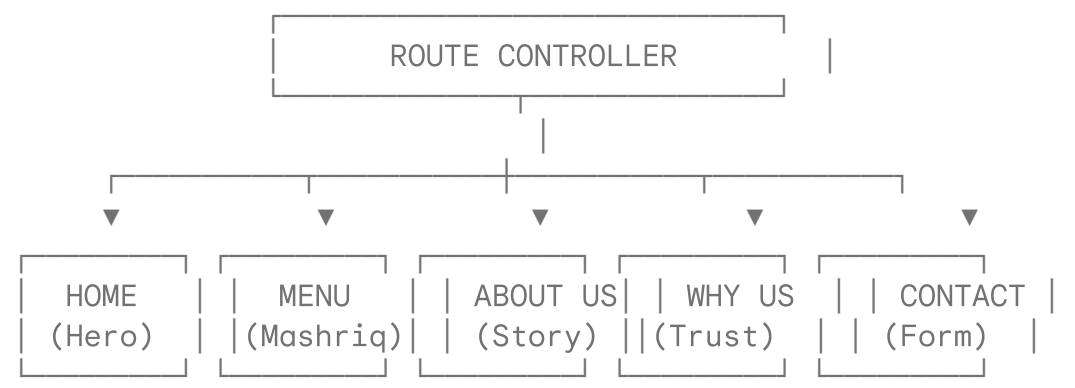
- **Behavior:** Manages physical floor seating in a fast-paced environment.
- **Need:** Instant notifications appearing on their host screen the moment a table is claimed.

2.3 The Restaurant Owner / Operator (Admin Console)

- **Behavior:** Needs to adjust menu prices dynamically based on seasonal ingredient costs or sweet/dessert inventory fluctuations.
- **Need:** A secure, web-accessible dashboard to manage menu items and monitor booking capacity.

3. Site Navigation & Page Architecture

The platform is designed around a seamless 5-page routing architecture optimized for customer trust and conversion.



3.1 Page 1: Home (The Landing Epicenter)

- **Dynamic Visual Hero Layer:** High-fidelity, atmospheric loop of the restaurant's Instagram culinary videos, optimized as highly compressed web assets.
- **Dual CTAs:**
 1. [Book a Table] (Primary CTA: Immediate reservation overlay/route).
 2. [Explore Sides of Mashriq] (Secondary CTA: Transitions user directly to the immersive, interactive menu).

3.2 Page 2: Explore Sides of Mashriq (The Interactive Menu)

- **UI Layout:** Tabbed category switcher (Italian, Arabic, Desi, Sweets & Desserts).
- **Responsive Grid:** Fluid CSS layouts adapting seamlessly between fast-food styling (bold pricing, speed tags) and fine-dining aesthetics (rich descriptions, wine/beverage pairing suggestions).

3.3 Page 3: About Us (The Fusion Backstory)

- **Backstory Content:** Captures the story behind fusing three distinct continents of flavor.

- **Visual Elements:** Highlight-reels and interactive timeline maps detailing the journey from Italian micro-kitchens to the spice-routes of Mashriq.

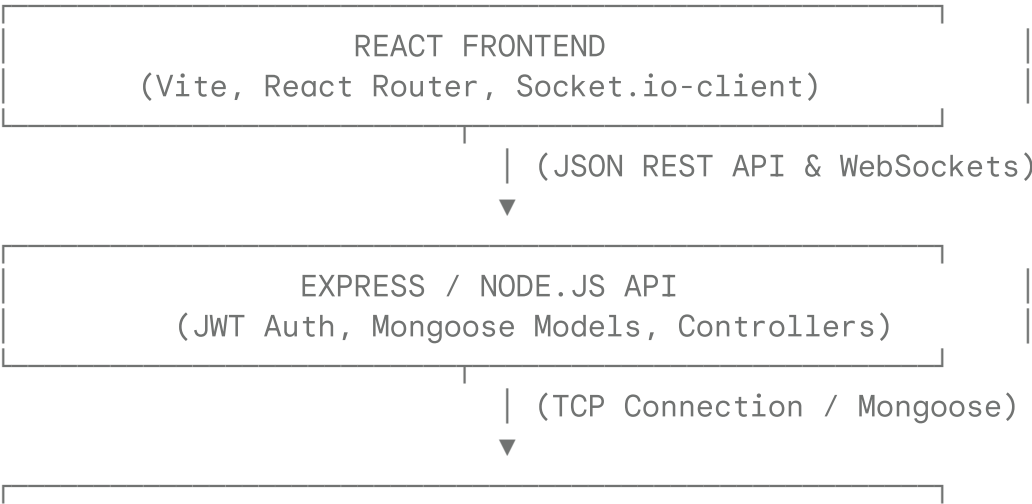
3.4 Page 4: Why Us (Value Proposition & Testimonials)

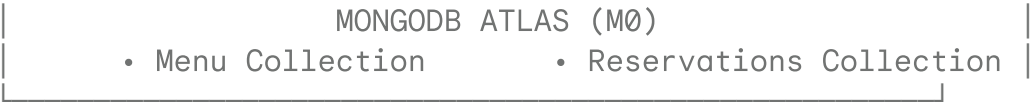
- **Core Value Props:** Spotlights premium, organic ingredient sourcing, zero markup fees, and the dual casual/fine-dining environment.
- **Testimonials Slider:** Pulls styled customer feedback cards, integrated with Instagram profile links to show raw social proof.

3.5 Page 5: Contact Us & Inquiries

- **Details:** Address, phone, hours of operation, and an interactive contact form.
- **Admin Routing:** Messages submitted through this page are saved to an `inquiries` collection in MongoDB for FOH review.

4. Technical Architecture (MERN Stack)





4.1 Real-Time Notifications

By shifting to the MERN stack, we use **Socket.io** to establish a bidirectional persistent TCP connection between the React client and Node.js backend.

- When a guest submits a booking request, the server emits a `newReservation` event.
- The host dashboard intercepts the socket message in sub-second speed:

Alert: Room T reserved for X guests.

4.2 Database Infrastructure (MongoDB Atlas Free Tier)

We utilize a shared free tier (M0) sandbox. Optimization is maintained via strict index placement to keep memory usage under the **512MB** ceiling:

- Single index on `reservations.dateTime` to optimize date-range queries.
- Compound index on `menu.cuisine` and `menu.category` for rapid category switching.

5. Database Schema Design (Mongoose)

5.1 Menu Model

```
const mongoose = require('mongoose');
```

```
const MenuSchema = new mongoose.Schema({
  cuisine: { type: String, enum: ['italian', 'arabic', 'desi', 'sweets'], required: true },
  category: { type: String, default: 'mains' },
  title: { type: String, required: true },
  description: { type: String },
  price: { type: Number, required: true },
  imageUrl: { type: String },
  isAvailable: { type: Boolean, default: true }
}, { timestamps: true });

module.exports = mongoose.model('Menu', MenuSchema);
```

5.2 Reservation Model

```
const mongoose = require('mongoose');

const ReservationSchema = new mongoose.Schema({
  customerName: { type: String, required: true },
  phoneNumber: { type: String, required: true },
  dateTime: { type: Date, required: true },
  guestCount: { type: Number, required: true, min: 1 },
  tableCategory: { type: String, enum: ['casual', 'fine-dining'], default: 'casual' },
  status: { type: String, enum: ['pending', 'confirmed', 'cancelled'], default: 'pending' }
}, { timestamps: true });

module.exports = mongoose.model('Reservation', ReservationSchema);
```

6. Non-Functional Performance Thresholds

- **Largest Contentful Paint (LCP):** Under 1.8 seconds on low-tier mobile networks.

- **Cumulative Layout Shift (CLS):** Absolute 0.0 threshold achieved by forcing explicit layout dimensions on dynamic elements.
- **Database Sync Speed:** Sub-second latency ($\leq 800\text{ms}$) on reservation writes and administrative alerts via Express API.